

## API Testing Evidence

Contains screenshots of Flask startup, the health-check page, phishing and legitimate API responses, LIME output and input-validation testing.

```
Predictions: C:\Users\sujan\Downloads\phishing-project\evaluation\results\unseen_predictions.csv
Metrics: C:\Users\sujan\Downloads\phishing-project\evaluation\results\unseen_metrics.json
Confusion matrix image: C:\Users\sujan\Downloads\phishing-project\evaluation\results\confusion_matrix.png
(.venv) PS C:\Users\sujan\Downloads\phishing-project> python app.py
Loading saved model...
C:\Users\sujan\Downloads\phishing-project\.venv\Lib\site-packages\torch\jit\_script.py:1485: FutureWarning: `torch.jit.script` is not supported in Python 3.14+ and may break. Please switch to `torch.compile` or `torch.export`.
  warnings.warn(
Saved model loaded successfully.
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
█
```

Fig1: Flask backend and saved model started successfully.

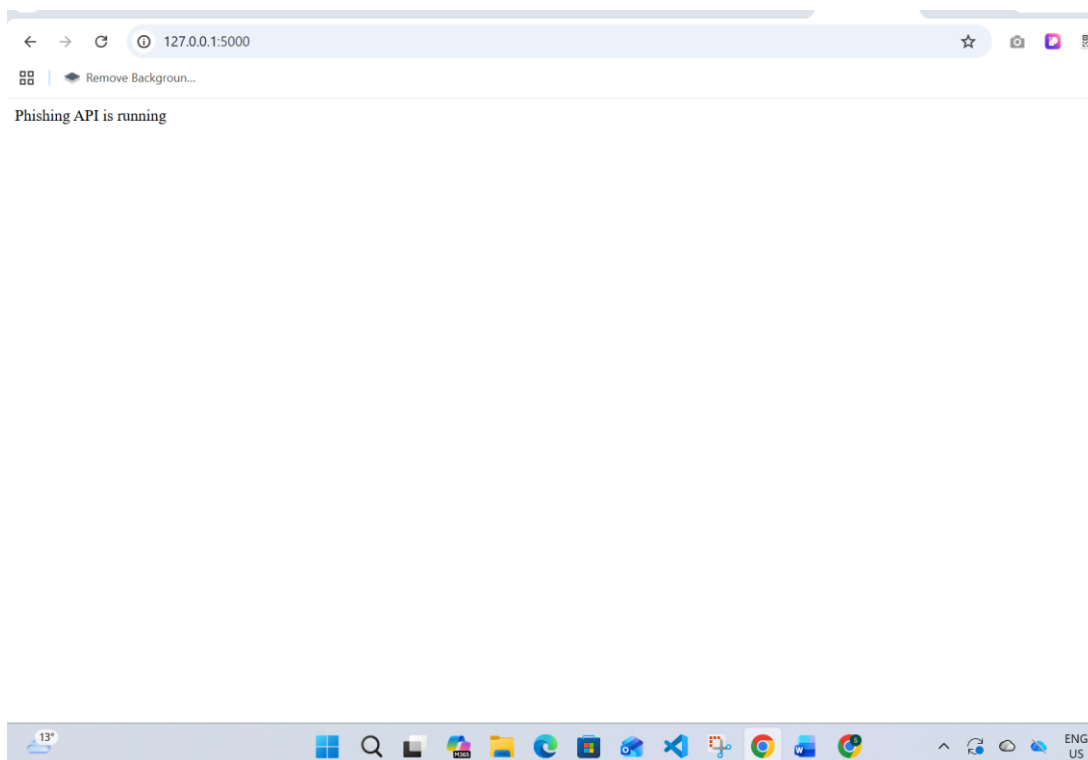
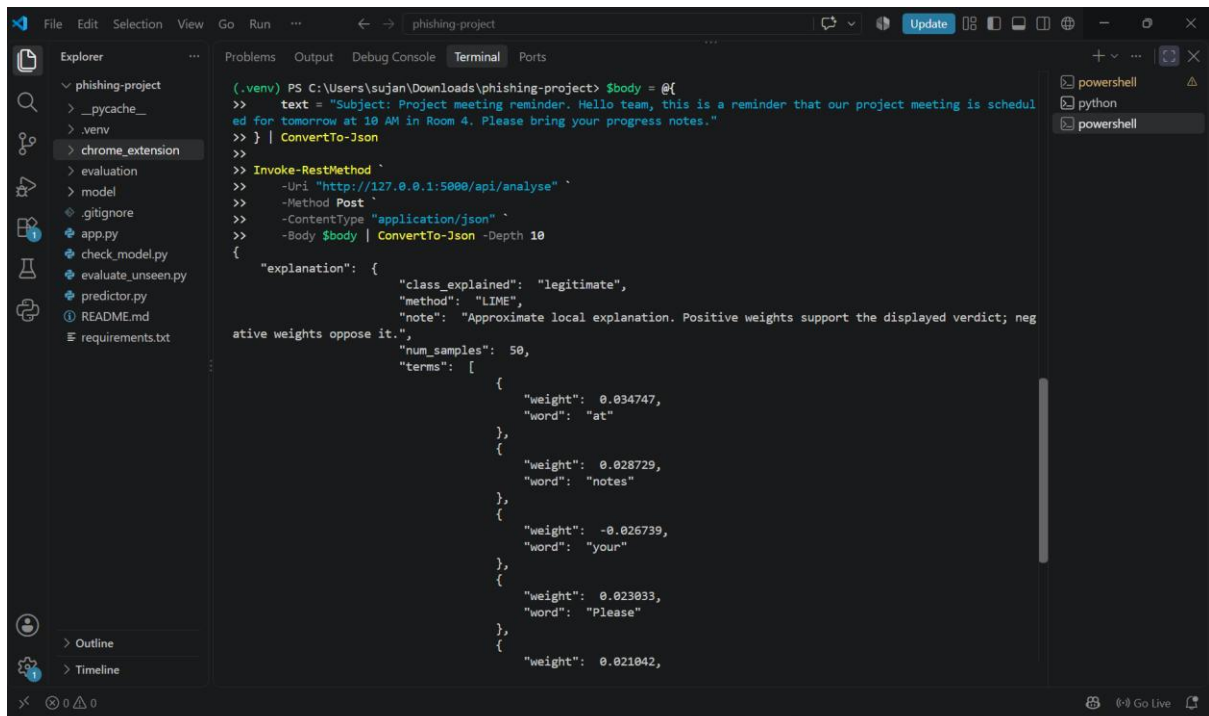


Fig 2: GET / check confirming that the Flask service is running.

```
PS C:\Users\sujan\Downloads\phishing-project> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned); (& c:\Users\sujan\Downloads\phishing-project\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\sujan\Downloads\phishing-project> $body = @{"text": "Subject: Account verification required. Unusual activity was detected on your account. Verify your password immediately at https://account-check.example.invalid/login or your account will be suspended."} | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://127.0.0.1:5000/api/analyse" -Method Post -ContentType "application/json" -Body $body | ConvertTo-Json -Depth 10
{"explanation": {"class_explained": "phishing", "method": "LIME", "note": "Approximate local explanation. Positive weights support the displayed verdict; negative weights oppose it.", "num_samples": 50, "terms": [{"weight": 0.265256, "word": "detected"}, {"weight": 0.178729, "word": "required"}, {"weight": -0.164749, "word": "account"}, {"weight": 0.094, "word": "activity"}]}}
```

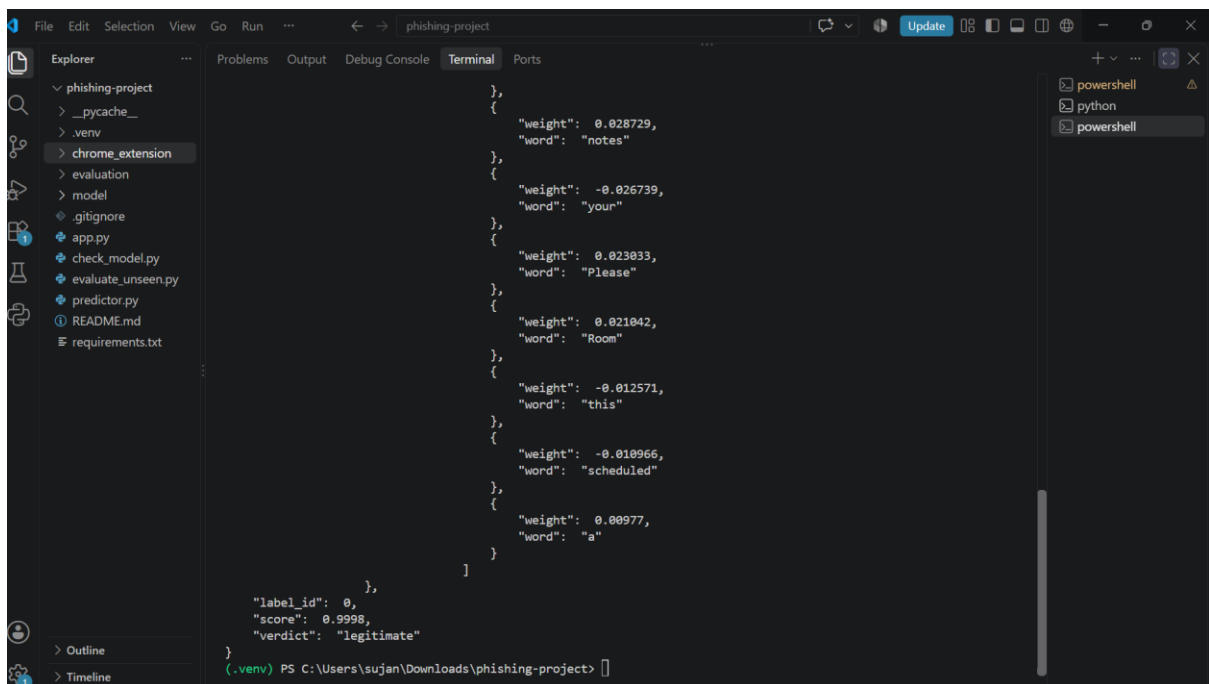
```
}, {"weight": 0.178729, "word": "required"}, {"weight": -0.164749, "word": "account"}, {"weight": 0.094, "word": "activity"}, {"weight": -0.07803, "word": "https"}, {"weight": -0.061648, "word": "at"}, {"weight": -0.044744, "word": "password"}, {"weight": 0.004682, "word": "will"}], "label_id": 1, "score": 0.9959, "verdict": "phishing"}
(.venv) PS C:\Users\sujan\Downloads\phishing-project>
```

Fig 3: Phishing API response containing the verdict, model score and LIME explanation.



The screenshot shows a Visual Studio Code editor with a REST client request defined in a file named `chrome_extension`. The request is a POST to `http://127.0.0.1:5000/api/analyse` with a JSON body. The response is partially visible, showing the `explanation` field.

```
(.venv) PS C:\Users\sujan\Downloads\phishing-project> $body = @{
>>   text = "Subject: Project meeting reminder. Hello team, this is a reminder that our project meeting is scheduled for tomorrow at 10 AM in Room 4. Please bring your progress notes."
>> } | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://127.0.0.1:5000/api/analyse" -Method Post -ContentType "application/json" -Body $body | ConvertTo-Json -Depth 10
{
  "explanation": {
    "class_explained": "legitimate",
    "method": "LIME",
    "note": "Approximate local explanation. Positive weights support the displayed verdict; negative weights oppose it.",
    "num_samples": 50,
    "terms": [
      {
        "weight": 0.034747,
        "word": "at"
      },
      {
        "weight": 0.028729,
        "word": "notes"
      },
      {
        "weight": -0.026739,
        "word": "your"
      },
      {
        "weight": 0.023033,
        "word": "Please"
      },
      {
        "weight": 0.021042,
```



This block continues the JSON response from the previous screenshot, showing the `terms` array and the final verdict and score.

```
    {
      "weight": 0.028729,
      "word": "notes"
    },
    {
      "weight": -0.026739,
      "word": "your"
    },
    {
      "weight": 0.023033,
      "word": "Please"
    },
    {
      "weight": 0.021042,
      "word": "Room"
    },
    {
      "weight": -0.012571,
      "word": "this"
    },
    {
      "weight": -0.010966,
      "word": "scheduled"
    },
    {
      "weight": 0.00977,
      "word": "a"
    }
  ],
  "label_id": 0,
  "score": 0.9998,
  "verdict": "legitimate"
}
(.venv) PS C:\Users\sujan\Downloads\phishing-project>
```

Fig 4: Legitimate API response containing the verdict, model score and LIME explanation.

```
(.venv) PS C:\Users\sujan\Downloads\phishing-project> python -c "import requests; r=requests.post('http://127.0.0.1:5000/api/analyse', json={'text':'a'*30001}); print('Status:', r.status_code); print(r.json())"
Status: 413
{'error': 'Email text must not exceed 30000 characters'}
```

Fig 5: HTTP 413 response produced when the input exceeds 30,000 characters.

```
(.venv) PS C:\Users\sujan\Downloads\phishing-project> try {
    Invoke-WebRequest `
    -Uri "http://127.0.0.1:5000/api/analyse" `
    -Method POST `
    -ContentType "application/json" `
    -Body '{"text":""}'
} catch {
    Write-Host "Status:" ([int]$_Exception.Response.StatusCode)
    Write-Host "Response:" $_.ErrorDetails.Message
}
Status: 400
Response: {"error":"Provide non-empty email text"}
```

Fig 6: HTTP 400 response produced when the request does not contain email text.

```
(.venv) PS C:\Users\sujan\Downloads\phishing-project> $body = @{}
(.venv) PS C:\Users\sujan\Downloads\phishing-project> Invoke-WebRequest -Uri http://127.0.0.1:5000/api/analyse -Method POST -ContentType application/json -Body $body
Status: 200
Response: {"text":"S1. Hello Sujana,\n\nYou need to visit a Service NSW Centre to finish your NDIS Worker Check application.\nIf you do not finish your application by 13 Oct 2026, it will be cancelled.\nApplication number: 789659045\nService NSW transaction number: 12345689\nHow to finish your application\nAt the Service NSW Centre, you need to:\n- confirm your identity\n- have your photo taken\n- pay the application fee.\nYou cannot do this at a local council agency.\nWhat to bring to the Service NSW Centre\nPlease bring the following documents as hard copies:\n- Driver License - NSW\n- Australian Visa\n- Bank card\n\nYou can also show this email at the Service NSW Centre.\nThe application number, transaction number and barcode help staff find your application and complete your request more quickly.\nYour application will then be lodged with the Office of the Children's Guardian for processing.\nTo learn more go to: https://www.ocg.nsw.gov.au/NDISWC.\nFor help, email contact@ndiswc.nsw.gov.au.\nTo find your nearest service centre, go to: https://www.service.nsw.gov.au/service-centre.\nHelp us improve by answering some questions about your experience using the NDIS Worker Check online form:\nhttps://forms.office.com/Pages/ResponsePage.aspx?id=aHr5Hqvo7UShbbV5_1182KjXpGr6K2RGtyo5HtVQ3L9UMk9KWESHNEVUMJhFMzNZVUJ3M1AwJEU5Mi4u.\nThank you,\nService NSW"}
(.venv) PS C:\Users\sujan\Downloads\phishing-project> ConvertTo-Json $body
{"text":""}
(.venv) PS C:\Users\sujan\Downloads\phishing-project> Invoke-WebRequest -Uri http://127.0.0.1:5000/api/analyse -Method POST -ContentType application/json -Body $body
Status: 400
Response: {"error":"Provide non-empty email text"}
```

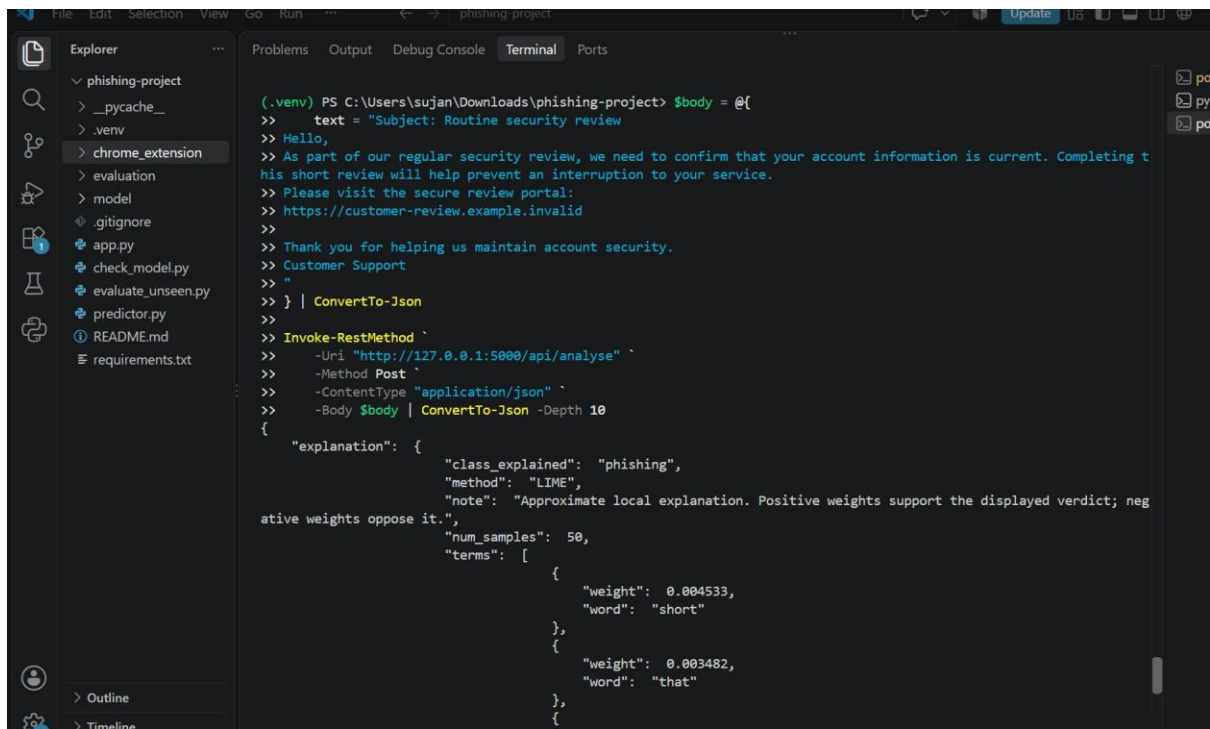
This screenshot shows the VS Code interface with a project named 'phishing-project'. The Explorer sidebar on the left lists files including '\_pycache\_', '.venv', 'chrome\_extension', 'evaluation', 'model', '.gitignore', 'app.py', 'check\_model.py', 'evaluate\_unseen.py', 'predictor.py', 'README.md', and 'requirements.txt'. The main editor window displays a JSON object representing the model's output for a legitimate email. The JSON structure includes an 'explanation' field with a 'class\_explained' of 'legitimate', a 'method' of 'LIME', and a 'note' explaining that positive weights support the verdict. A list of 'terms' with their respective weights and words is also provided.

```
{
  "explanation": {
    "class_explained": "legitimate",
    "method": "LIME",
    "note": "Approximate local explanation. Positive weights support the displayed verdict; negative weights oppose it.",
    "num_samples": 50,
    "terms": [
      {
        "weight": 0.296871,
        "word": "the"
      },
      {
        "weight": 0.242881,
        "word": "12345689"
      },
      {
        "weight": 0.217584,
        "word": "it"
      },
      {
        "weight": 0.197348,
        "word": "Sujina"
      },
      {
        "weight": 0.171198,
        "word": "Help"
      },
      {
        "weight": -0.133447,
        "word": "be"
      },
      {
        "weight": 0.130427,
        "word": "to"
      }
    ]
  }
}
```

This screenshot shows the same VS Code interface as the first image, but with a more complete JSON output. The 'terms' list is identical, but it is followed by a 'label\_id' of 0, a 'score' of 0.9998, and a 'verdict' of 'legitimate'. The terminal at the bottom shows the command prompt for the '.venv' environment.

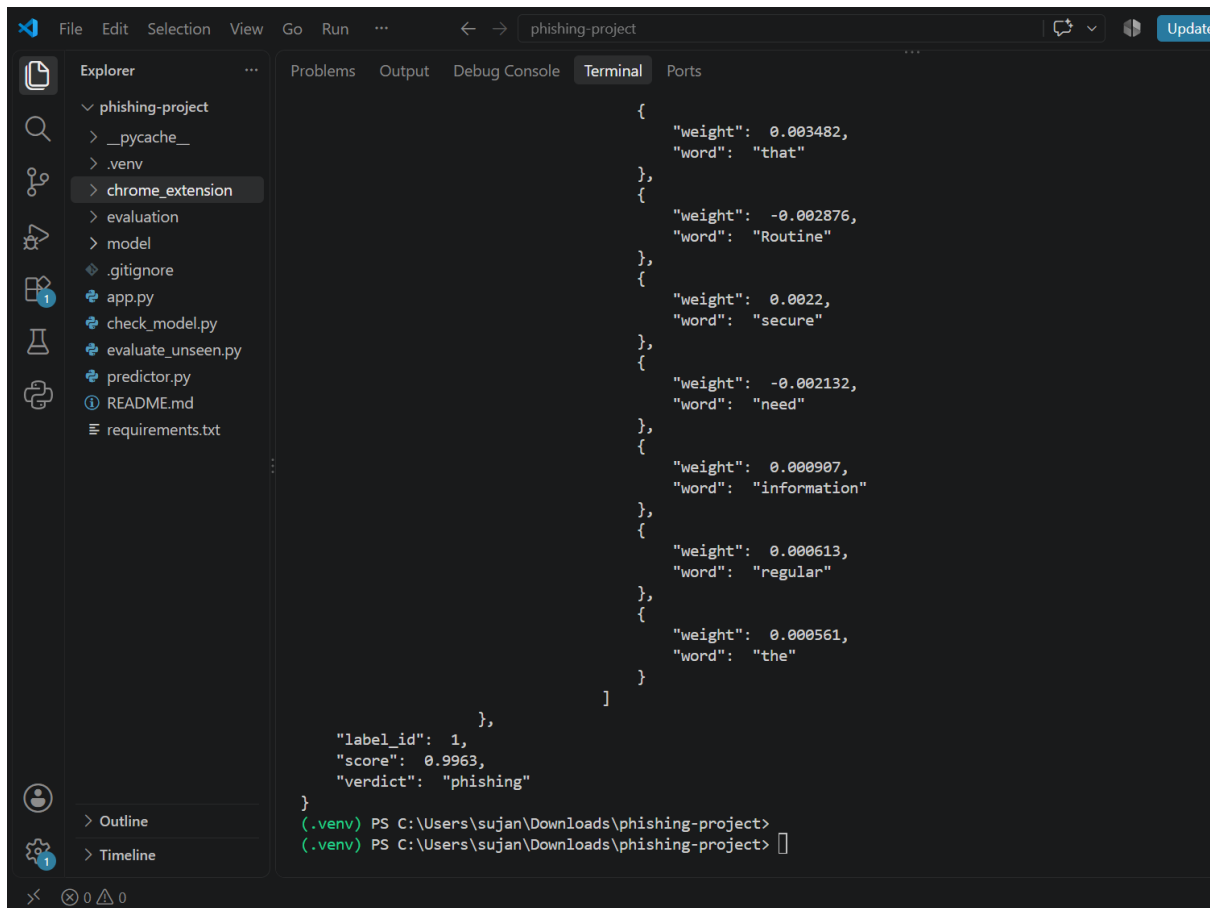
```
    ],
    {
      "weight": 0.242881,
      "word": "12345689"
    },
    {
      "weight": 0.217584,
      "word": "it"
    },
    {
      "weight": 0.197348,
      "word": "Sujina"
    },
    {
      "weight": 0.171198,
      "word": "Help"
    },
    {
      "weight": -0.133447,
      "word": "be"
    },
    {
      "weight": 0.130427,
      "word": "to"
    },
    {
      "weight": -0.012029,
      "word": "nearest"
    }
  ],
  "label_id": 0,
  "score": 0.9998,
  "verdict": "legitimate"
}
(.venv) PS C:\Users\sujan\Downloads\phishing-project>
```

Fig 7: Another email tested which is legitimate



The screenshot shows a VS Code window with a terminal. The Explorer sidebar on the left shows a project named 'phishing-project' with files like 'chrome\_extension', 'evaluation', 'model', '.gitignore', 'app.py', 'check\_model.py', 'evaluate\_unseen.py', 'predictor.py', 'README.md', and 'requirements.txt'. The terminal is running a PowerShell command to invoke a REST client. The command sets a body with a phishing email text and sends a POST request to 'http://127.0.0.1:5000/api/analyse' with a content type of 'application/json'. The response is a JSON object with an 'explanation' field containing a LIME analysis.

```
(.venv) PS C:\Users\sujan\Downloads\phishing-project> $body = @{
>>   text = "Subject: Routine security review
>>   Hello,
>>   As part of our regular security review, we need to confirm that your account information is current. Completing t
his short review will help prevent an interruption to your service.
>>   Please visit the secure review portal:
>>   https://customer-review.example.invalid
>>
>>   Thank you for helping us maintain account security.
>>   Customer Support
>> } | ConvertTo-Json
>>
>> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/api/analyse" `
>>   -Method Post `
>>   -ContentType "application/json" `
>>   -Body $body | ConvertTo-Json -Depth 10
{
  "explanation": {
    "class_explained": "phishing",
    "method": "LIME",
    "note": "Approximate local explanation. Positive weights support the displayed verdict; neg
ative weights oppose it.",
    "num_samples": 50,
    "terms": [
      {
        "weight": 0.004533,
        "word": "short"
      },
      {
        "weight": 0.003482,
        "word": "that"
      }
    ]
  }
}
```



This screenshot shows the continuation of the terminal output from the previous image. It displays the full JSON response, including the 'label\_id', 'score', and 'verdict'. The 'explanation' field contains a list of terms with their weights, such as 'that', 'Routine', 'secure', 'need', 'information', 'regular', and 'the'. The final verdict is 'phishing' with a score of 0.9963.

```
    },
    "label_id": 1,
    "score": 0.9963,
    "verdict": "phishing"
  }
}
(.venv) PS C:\Users\sujan\Downloads\phishing-project>
(.venv) PS C:\Users\sujan\Downloads\phishing-project>
```

Fig 8: Phishing email tested